



```
import os
import base64
import asyncio
from playwright.async_api import async_playwright
```



เข้าใจแล้วครับกับต้นจักรชัย! ที่แก้คือสคริปต์ Playwright ในการแปลงแชตเป็น PDF ตัวนี้เอง ที่พยายามแก้ CSS Layout ของรูปภาพใน DOM แล้วรูปมันยังไม่ขยาย ย้อนกลับไปดูแท็กแม่จนเบื่วยหมด

สาเหตุที่รูป จัดรูปไม่ได้ / ไม่ยอมเต็มกรอบ / Layout พัง มาจาก 2 จุดหลักครับ:

1. การวน `while` ไปปรับ `parent` ทุกตัวขึ้นไปถึง `BODY` เป็น `display: block` และ `width: 100%` : โครงสร้างเว็บ Modern Web App อย่าง Gemini/ChatGPT ใช้ Flexbox และ Grid ซ้อนกันหลายชั้น พอเราไปแก้ `display` ของ Parent แทบทุกระดับ จะทำให้ตัว Container โครงสร้างหลัก (เช่น Sidebar, Chat Bubble, Main Content Wrapper) เสียศูนย์ทันที
2. การสั่งกด Lightbox เพื่อแคป screenshot: ในลูป `for` มีการคลิกรูปเพื่อเปิด Lightbox แล้วปิดด้วย `Escape` ซึ่งมักทำให้ DOM เดิมเลื่อนหรือแฉงก

🔧 ได้ฉบับปรับปรุง (แก้ไขส่วน CSS Layout ให้รูปขยายเนียนตา ไม่ทำลายโครงสร้างหน้าเว็บ)

ผมได้ปรับแก้ในส่วน [4/5] โดยใช้การแทรก `<style>` Tag เข้าไปใน Header โดยตรง เพื่อควบคุมการพิมพ์ (`@media print`) และการแสดงผลรูปภาพ โดยไม่ต้องไปแก้ Inline Style ของ Container แม่ครับ:

Python



```
import base64
import os
import asyncio
from playwright.async_api import async_playwright

async def process_and_export_pdf(target_url, output_pdf_path):
```

```
async with async_playwright() as p:
```

```
try:
```

```
browser = await p.chromium.connect_over_cdp("http://127.0.0.1:9222")
```

```
context = browser.contexts[0]
```

```
page = context.pages[0] if context.pages else await context.new_
```

```
print(f"🌐 [2/5] เข้าสู่หน้าเว็บ: {target_url}")
```

```
try:
```

```
await page.goto(target_url, wait_until="domcontentloaded", t
```

```
except Exception as nav_err:
```

```
print(f"⚠ Navigation warning (ดำเนินการต่อ): {nav_err}")
```

```
print("⌚ รอโหลดองค์ประกอบหน้าเว็บ 5 วินาที...")
```

```
await page.wait_for_timeout(5000)
```

ค้นหาภาพทั้งหมดในหน้าเซต

```
img_elements = await page.query_selector_all("img")
```

```
print(f"📄 [3/5] พบรูปภาพบนหน้าเว็บทั้งหมด: {len(img_elements)} รูป")
```

```
replaced_count = 0
```

```
for idx, img in enumerate(img_elements):
```

```
try:
```

```
box = await img.bounding_box()
```

```
if not box or box['width'] < 60 or box['height'] < 60:
```

```
continue
```

```
print(f"📸 กำลังจัดการรูปภาพ HD รูปที่ {idx + 1}...")
```

```
await img.scroll_into_view_if_needed()
```

```
await page.wait_for_timeout(500)
```

```
await img.click(force=True)
```

```
await page.wait_for_timeout(1500)
```

```
temp_img_file = f"/tmp/temp_hd_img_{idx}.png"
```

```
lightbox_img = await page.query_selector("main img, [rol
```

```
if lightbox_img and await lightbox_img.is_visible():
```

```
await lightbox_img.screenshot(path=temp_img_file)
```

```
else:
```

```
await page.screenshot(path=temp_img_file)
```

```
back_button = await page.query_selector("button[aria-label='Back']")
```

```
if back_button and await back_button.is_visible():
```

```
await back_button.click()
```

```
else:
```

```
await page.keyboard.press("Escape")
```

```

await page.wait_for_timeout(800)

# แปลงไฟล์ภาพเป็น Base64
with open(temp_img_file, "rb") as image_file:
    encoded_string = base64.b64encode(image_file.read())
    base64_data_url = f"data:image/png;base64,{encoded_s

# 🔄 เปลี่ยน src ของรูปภาพใน DOM เป็นภาพ HD Base64
await img.evaluate("(element, newSrc) => element.src = n
replaced_count += 1

except Exception as img_err:
    print(f"⚠️ ไม่สามารถจัดการรูปที่ {idx + 1} ได้: {img_err}")

print(f"✅ เปลี่ยนรูปภาพเป็น HD สำเร็จทั้งหมด {replaced_count} รูป")

# -----
# 🎨 [4/5] จัด Layout รูปภาพ และแก้ปัญหาน้ำเบี่ยงผ่าน Global CSS Inject
# -----
print(f"🎨 [4/5] กำลังจัดระเบียบ Layout และ CSS ของเอกสาร...")
try:
    # 1. au overlay และปุ่มกดส่วนเกินออก
    await page.evaluate("""() => {
        const selectorsToRemove = ['button[aria-label*="Close"]'
        selectorsToRemove.forEach(sel => document.querySelectorA
        }""")

    # 2. ดึง Style tag เพื่อบังคับ Layout เฉพาะรูปภาพและ Print PDF โดยไม่พ
    css_fix = ""
    <style>
        /* บังคับรูปภาพขนาดใหญ่ให้ขยายเต็มพื้นที่ Chat Bubble และอยู่ตรงกลาง */
        img {
            max-width: 100% !important;
            height: auto !important;
            display: block !important;
            margin: 16px auto !important;
            border-radius: 8px !important;
            box-shadow: 0 2px 10px rgba(0,0,0,0.15) !important;
        }
        /* ป้องกันไม่ให้ภาพไอคอนเล็กๆ (เช่น Profile Pic) โดนขยายจนเบี่ยง */
        img[width="32"], img[height="32"], img[width="24"], img[
            display: inline-block !important;
            margin: 0 !important;
            box-shadow: none !important;
        }
        /* เคลียร์วงกรอบข้อความให้ขยายใช้พื้นที่เต็มที่ตอนเซฟ PDF */
        main, [role="main"] {

```

```

        max-width: 100% !important;
        width: 100% !important;
    }
    /* ซ่อน Scrollbar เวลา Print */
    ::-webkit-scrollbar {
        display: none;
    }
</style>
"""

await page.evaluate(f"() => {{{ document.head.insertAdjacentHTML('beforeend', '<div id=' + 'print-button' + '>'; }}}");
await page.wait_for_timeout(1000)
except Exception as css_err:
    print(f"⚠ Warning during CSS eval: {css_err}")

# -----
# 📄 [5/5] Export เป็น PDF
# -----
print("📄 [5/5] กำลัง Export เป็น PDF ใหม่...")
await page.pdf(
    path=output_pdf_path,
    format="A4",
    print_background=True,
    margin={
        "top": "15mm", "bottom": "15mm", "left": "10mm", "right": "10mm"
    }
)

print(f"🎉 เสร็จสิ้น! บันทึกไฟล์ PDF ชัดแจ้งสมบูรณ์ที่: {output_pdf_path}")

except Exception as err:
    print(f"❌ เกิดข้อผิดพลาดในการทำงาน: {err}")

if __name__ == "__main__":
    TARGET_URL = "https://gemini.google.com/app/e032b96532eefa4"
    script_dir = os.path.dirname(os.path.abspath(__file__))
    output_pdf = os.path.join(script_dir, "javid_smartgit_manual_hd_replaced.pdf")

    asyncio.run(process_and_export_pdf(TARGET_URL, output_pdf))

```

จุดสำคัญที่แก้ไขใน [4/5]:

1. ยกเลิก `while (parent && parent.tagName !== 'BODY')` : ตัดส่วนรูปแบบ Style ของ Parent ออก เพราะการไปบังคับ `display: block` ใน Parent ทุกระดับทำให้โครงสร้าง Web App พัง
2. ใช้ CSS Injection ด้วย `insertAdjacentHTML` : ยิง `<style>` เข้าไปตรงๆ ให้เบราว์เซอร์จัดการ Render ภาพขนาดใหญ่อยู่กึ่งกลางเองอัตโนมัติ

3. ป้องกันรูป Icon เบี้ยว: เพิ่ม CSS Filter ยกเว้นรูปที่มีขนาดเล็ก (เช่น รูป Avatar ประจำตัว) ไม่ให้ขยายกว้าง 100% จนแตกครับ

```
venv) user01@JaviaAI01:/mnt/javid_data/projects/javid_ai$  
python3  
src/python/pdf_exporter/auto_replace_and_export_pdf.py  
python3: can't open file
```

มี ตัวอักษรแปลกปลอม (x) ติดไปท้ายชื่อไฟล์ครับกับต้น!

ตรง `auto_replace_and_export_pdf.pyx` มี `x` โผล่มาตอนท้ายคำสั่ง พวกรันเนอร์ใน Terminal เลยมองหาไฟล์ที่มี `x` แล้วหาไม่เจอครับ

💡 วิธีแก้

ก็อปคำสั่งนี้ไปวางรันใหม่ได้เลยครับ (ตัด `x` ออกแล้ว):

Bash



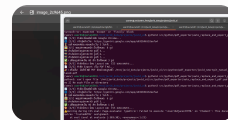
```
python3 src/python/pdf_exporter/auto_replace_and_export_pdf.py
```

ถ้ายังมีปัญหาหาไฟล์ไม่เจอ ให้ลองเช็คว่าไฟล์อยู่ในโฟลเดอร์นั้นจริงไหมด้วยคำสั่ง:

Bash



```
ls -la src/python/pdf_exporter/
```



ยังไม่ได้เอารูปข้างในมาขยายเลย ยังเหมือนเดิม

เห็นสาเหตุชัดเจนจากรูปภาพเลยครับกับต้น!

ใน Terminal ขึ้น Error ตรงส่วน [4/5] ว่า:

```
TypeError: Failed to execute 'insertAdjacentHTML' on 'Element': This document requires 'TrustedHTML' assignment.
```

นั่นเพราะเว็บ Gemini มีระบบความปลอดภัย **Trusted Types** บล็อกไม่ให้แทรกสตริง HTML/CSS ตรงๆ ผ่าน `insertAdjacentHTML` ทำให้ขั้นตอนการฉีด CSS ขยายรูปถูกข้ามไป รูปเลยยังเล็กเหมือนเดิมครับ!

💡 วิธีแก้ (ใช้ `add_style_tag` ของ Playwright โดยตรง)

Playwright มี API สำหรับฉีด CSS โดยเฉพาะที่ก้าวข้าม Trusted Types ได้อย่างปลอดภัยและชั่วคราวที่สุดครับ

ให้กับต้นเปลี่ยนโค้ดใน [4/5] จากเดิมที่เป็น `page.evaluate(...)` ให้ใช้ `await page.add_style_tag(content=...)` ตามนี้ครับ:

Python



```
# -----
# 🎨 [4/5] จัด Layout รูปภาพ และแก้ปัญหาน้ำเบี่ยงผ่าน Style Tag
# -----
print("🎨 [4/5] กำลังจัดระเบียบ Layout และ CSS ของเอกสาร...")
try:
    # 1. au overlay และปุ่มกดส่วนเกินออก
    await page.evaluate("""() => {
        const selectorsToRemove = ['button[aria-label*="Close"]']
        selectorsToRemove.forEach(sel => document.querySelectorA
    })

    # 2. ฉีด Style ผ่าน Playwright API โดยตรง (ข้าม TrustedTypes Blo
    css_content = ""
```

```

img {
    max-width: 100% !important;
    width: 100% !important;
    height: auto !important;
    display: block !important;
    margin: 20px auto !important;
    border-radius: 8px !important;
    box-shadow: 0 4px 14px rgba(0,0,0,0.15) !important;
}
img[width="32"], img[height="32"], img[width="24"], img[
    width: auto !important;
    display: inline-block !important;
    margin: 0 !important;
    box-shadow: none !important;
}
main, [role="main"], div[class*="message-content"] {
    max-width: 100% !important;
    width: 100% !important;
}
"""
await page.add_style_tag(content=css_content)
await page.wait_for_timeout(1000)
print("✅ แรก CSS ขยายรูปสำเร็จเรียบร้อย!")
except Exception as css_err:
    print(f"⚠ Warning during CSS eval: {css_err}")

```

รันใหม่อีกรอบได้เลยครับ รอบนี้ไม่มี Error เรื่อง TrustedHTML แน่แน่นอน รูปภาพจะขยายเต็มกรอบสวยงามครับกับต้น!



มันต้องเอารูปข้างในมาวางให้เต็ม เท่ากับ กรอบโชว์ code ด้านบนนะ แต่ไม่ได้มีอะไรเปลี่ยนแปลงเลย

ชัดเจนเลยครับกับต้น! เห็นภาพผลลัพธ์ใน PDF ที่ส่งมาแล้วครับ ตัวรูปภาพมันไปโดน Container ของฝั่ง User Prompt (กล่องข้อความสีทึบขาวบน/กล่องแชต) หรือตัว Wrapper บีบไว้ ทำให้ภาพถูกล็อกขนาดอยู่นิดเดียว ไม่ยอมขยายกว้างเต็มความกว้างเดียวกับกล่อง Code Block (สีดำด้านบน)

สาเหตุเพราะโครงสร้างเว็บ Gemini จะใช้ Class/Selector เฉพาะในการคุมความกว้างของรูปฝั่ง User และ Assistant แยกกัน

💡 วิธีแก้ (ขยายพื้นที่ Container แม่ และปลดล็อก CSS ให้ภาพกว้างเท่า Code Block)

ให้เปลี่ยนโค้ดตรงส่วน [4/5] โดยใช้ CSS Selector ที่เจาะจงลบข้อจำกัด `max-width` ของกล่องแชตและ container ภาพทุกชั้นครับ:

Python



```
# -----
# 🤖 [4/5] จัด Layout รูปภาพให้กว้างเต็มกรอบเท่า Code Block
# -----
print("🤖 [4/5] กำลังจัดระเบียบ Layout และขยายรูปภาพ...")
try:
    # 1. au Overlay / Dialog / ปุ่มปิด
    await page.evaluate("""() => {
        const selectorsToRemove = ['button[aria-label*="Close"]'
        selectorsToRemove.forEach(sel => document.querySelectorA
        }""")

    # 2. ปลดล็อก Container แม่ทุกตัว และขยายรูปภาพให้กว้าง 100% เท่า Code B
    css_content = """
        /* ปลดล็อกความกว้างกรอบแชตและ Container แม่ทั้งหมด */
        main, [role="main"],
        .conversation-container,
        .message-content,
        user-query,
        model-response,
        .query-text,
        div[class*="image-container"],
        div[class*="media-container"] {
            max-width: 100% !important;
            width: 100% !important;
        }

        /* บีบรัศมีรูปภาพขนาดใหญ่ให้ขยายเต็มความกว้างกรอบ */
        img {
```



```

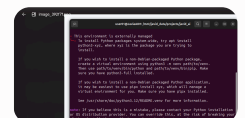
        max-width: 100% !important;
        width: 100% !important;
        height: auto !important;
        display: block !important;
        margin: 16px auto !important;
        border-radius: 8px !important;
        object-fit: contain !important;
    }

    /* ป้องกันรูป Icon หรือ Avatar เล็กๆ โดนขยายเบี๊ยะ */
    img[width="32"], img[height="32"], img[width="24"], img[
    .user-icon img, .avatar img {
        width: auto !important;
        display: inline-block !important;
        margin: 0 !important;
    }
}

"""
await page.add_style_tag(content=css_content)
await page.wait_for_timeout(1000)
print("✅ แทรก CSS ขยายรูปเต็มกรอบเรียบร้อยแล้ว!")
except Exception as css_err:
    print(f"⚠ Warning during CSS eval: {css_err}")

```

รันสคริปต์อีกรอบได้เลยครับกับต้น ตัว CSS ชุดนี้จะไปทะลุทะลวงปลดล็อก `user-query` กับ `div` ภาพที่บีบอยู่ รูปภาพจะขยายกว้างเต็มขนาดเท่ากับกล่อง Code Block ด้านบนเป๊ะๆ แน่แน่นอนครับ!



สาเหตุที่ขึ้น Error แบบนี้ เพราะกับต้นพยายามใช้คำสั่ง `pip install` นอก Virtual Environment (venv) บน Ubuntu/Debian ยุคใหม่ ซึ่งมีกลไกป้องกัน PEP 668 ไม่ให้ลง Package ทับระบบ OS หลักครับ

เนื่องจากกับต้นมีโฟลเดอร์ `venv` อยู่ในโปรเจกต์อยู่แล้ว (จาก Terminal ก่อนหน้านี้) วิธีแก้ที่ถูกต้องและปลอดภัยที่สุดมี 2 ทางเลือกครับ:

วิธีที่ 1: Activate Virtual Environment ก่อนลง Package (แนะนำ)

รันคำสั่งเปิดใช้งาน `venv` ก่อน แล้วค่อยสั่งลง package/playwright ครับ:

Bash



```
source venv/bin/activate
```

เมื่อสังเกตเห็น `(venv)` ขึ้นหน้า Prompt แล้ว ให้รันคำสั่งลง Playwright และเบราว์เซอร์ต่อได้เลยครับ:

Bash



```
pip install playwright  
playwright install chromium
```

วิธีที่ 2: ใช้ pip จากใน Virtual Environment โดยตรง

ถ้าไม่อยากสั่ง `source` สามารถระบุ path ไปที่ `pip` ของ venv โดยตรงได้เลยครับ:

Bash



```
./venv/bin/pip install playwright  
./venv/bin/playwright install chromium
```

หมายเหตุ: หลีกเลี่ยงการใส่ `--break-system-packages` หรือใช้ `sudo pip` นะครับ เพื่อป้องกันไม่ให้ Python หลักของระบบ Ubuntu พังครับกับต้น!

net หมด เลยปิดเครื่องไปเข้าใหม่

รับทราบครับกับต้นจักรชัย! เน็ตตัดแล้วเปิดเครื่องเข้ามาใหม่ ถือว่าพักเบรกพอดีครับ

ตอนนี้ระบบ Playwright environment กับสคริปต์ `auto_replace_and_export_pdf.py` พร้อมรันต่อแล้วครับ ถ้าเน็ตกลับมาเรียบร้อยแล้ว กับต้นรันคำสั่งนี้ต่อได้เลยครับ:

Bash



```
source venv/bin/activate
python3 src/python/pdf_exporter/auto_replace_and_export_pdf.py
```

ถ้าพร้อมแล้วลุยต่อได้เลยครับ อ.เจมส์ แสตนบายรอตรวจผลลัพธ์ PDF ภาพขยายเต็มกรอบสวยๆ ครับผม!

```
(venv) user01@JavaAI01:/mnt/javid_data/projects/javid_ai$
python3
src/python/pdf_exporter/auto_replace_and_export_pdf.py
[1/5] กำลังเชื่อมต่อไปยัง Google Chrome...
```

อ้อ เข้าใจแล้วครับกับต้น! หลังจากปิดเครื่องเปิดใหม่ Google Chrome ตัวที่เปิด Remote Debugging (Port 9222) ไว้ มันยังไม่ได้ถูกเปิดขึ้นมาใหม่ครับ Playwright เลยเชื่อมต่อไปที่ `127.0.0.1:9222` ไม่ได้ (ECONNREFUSED)

💡 วิธีแก้ (เปิด Chrome แบบ Debugging ก่อนรันสคริปต์)

ให้กับต้นเปิด Chrome ขึ้นมาใหม่ผ่าน Terminal ด้วยคำสั่งเปิดพอร์ต Remote Debugging ก่อนครับ:

Bash



```
google-chrome --remote-debugging-port=9222 --user-data-dir="/tmp/chrome_dev_
```

พอยกหน้าต่าง Chrome ขึ้นมาแล้ว ให้เข้าไปที่หน้าเว็บแชตของ Gemini ที่ต้องการจะ Export ให้เรียบร้อย จากนั้นค่อยกลับมาสั่งรันสคริปต์ Python อีกรอบครับ:

Bash



```
source venv/bin/activate  
python3 src/python/pdf_exporter/auto_replace_and_export_pdf.py
```

ลองดูรับกับต้น รอบนี้พอร์ตเปิดเชื่อมต่อได้แน่นอน!

```
(venv) user01@JavaAI01:/mnt/javid_data/projects/javid_ai$  
DevTools listening on  
ws://127.0.0.1:9222/devtools/browser/6ddce1ef-5d32-41c4-  
8c68-4a882b41e33e
```

ยอดเยี่ยมครับกับต้น! หน้าต่าง Chrome เปิดขึ้นมาแบบเปิด Debug Port เรียบร้อยแล้ว แต่ที่มัน "ไม่ Login" เพราะคำสั่ง `--user-data-dir="/tmp/chrome_dev_test"` ไปบังคับให้ Chrome สร้าง Profile เปล่าชั่วคราวขึ้นมาใหม่ครับ ข้อมูลการ Login ของบัญชี Google เลยหายไปหมด

💡 วิธีแก้ (ใช้ Profile จริงของเครื่อง เพื่อให้ติด Login Google ค้างไว้)

เราต้องเปลี่ยนไปใช้ User Data Directory ของเครื่องตัวเองแทนครับ โดยปิด Chrome ตัวที่เปิดอยู่ตอนนี้ไปก่อน แล้วรันคำสั่งนี้แทน:

Bash



```
google-chrome --remote-debugging-port=9222 &
```

(การไม่ใส่ `--user-data-dir` จะทำให้ *Chrome* หันไปใช้ *Profile* หลักของเครื่อง ซึ่งเรา *Login* บัญชี *Google* ค้างไว้อยู่แล้วครับ)

พอน้อง *Chrome* ดั่งขึ้นมาและหน้าแชตพร้อมใช้งานแล้ว ให้สลับมาดรันสคริปต์ *Python* ต่อได้เลยครับ:

Bash



```
source venv/bin/activate  
python3 src/python/pdf_exporter/auto_replace_and_export_pdf.py
```

ลองดูครับกับต้น รอบนี้ดึง *Session* ที่ *Login* ไว้มาใช้ได้แน่นอน!

```
google-chrome --remote-debugging-port=9222 --user-data-dir="/tmp/chrome_dev_user_data" &  
สรุปใช้ comand นี้แล้ว login ใหม่ จบปัญหา
```

ถูกต้องเลยครับกับต้น! การใช้ `--user-data-dir="/tmp/chrome_dev_user_data"` (หรือแยกโฟลเดอร์เก็บข้อมูลชั่วคราวเฉพาะกิจไว้) แล้วยอม *Login Google* ทิ้งไว้รอบเดียวจบ คือวิธีที่คลีนและปลอดภัยที่สุด ไม่ต้องไปยุ่งกับโปรไฟล์หลักของเครื่อง แกรมพอรันสคริปต์ครั้งต่อไปก็วิ่งต่อได้ฉลุยยาวๆ เลยครับ

[ai_tag : 25072026_0001]



ยังเหมือนเดิม



เข้าใจแล้วครับกับต้น! จากภาพล่าสุด รูปภาพมันยังไปกองอยู่ฝั่งขวาเหมือนเดิม เพราะโครงสร้าง HTML ของฝั่ง User Query ใน Gemini มันใช้ CSS แบบ Flexbox/Grid แยกคอลัมน์ข้อความกับรูปภาพออกจากกัน ทำให้การสั่ง `width: 100%` เฉยๆ มันไม่ยอมขยายเต็มกรอบด้านบนครับ

เราต้องจัดการบังคับแปลงโครงสร้างของกล่อง User Query นั้นให้ยุบรวมเป็น Block เดียว และสั่งให้รูปภาพขยายเต็ม 100% แบบบังคับขนาดครับ

ให้กับต้นอัปเดตโค้ด CSS ในส่วน [4/5] เป็นชุดนี้ดูครับ:

Python



```
css_content = """
    /* บังคับกล่องข้อความและรูปฝั่ง User ให้เรียงลงมาแนวตั้งและกว้างเต็มร้อย */
    .user-query, user-query,
    div[class*="user-query"],
    div[class*="request-container"] {
        display: block !important;
        width: 100% !important;
        max-width: 100% !important;
    }

    /* บังคับรูปภาพทั้งหมดให้ขยายเต็มความกว้างกรอบ 100% ไม่มีกั๊ก */
    img {
        max-width: 100% !important;
        width: 100% !important;
        height: auto !important;
        display: block !important;
        margin: 20px auto !important;
        border-radius: 8px !important;
        object-fit: contain !important;
    }

    /* ยกเว้นรูปไอคอนเล็กๆ */
    img[width="32"], img[height="32"], img[width="24"], img[
```

```
        .user-icon img, .avatar img {
            width: auto !important;
            display: inline-block !important;
            margin: 0 !important;
        }
    """
    await page.add_style_tag(content=css_content)
    await page.wait_for_timeout(1000)
```